

Theseus builds its own hands: the self-improvement (RSI) plan

Epic theseus-pw1q, a child of the north star theseus-3iwi. Written 2026-10-09 10:02 MST by Tabitha/Claude, from the code at origin/main 69cb5375 (~/projects/theseus) and the workspace's briefs and memories. Read-only: no code changed, no model called. Amended 10:20 the same day with the owner's decision: act first, tell him after (section 0). The reports and briefs it cites are the working investigations, kept outside this repository.

The answer first

What RSI means for Theseus, in plain words. Recursive self-improvement here is not a model rewriting its own weights. The model stays as it is. What improves is everything around it: the **harness** grows its own **knowledge, judgement, tools, compute hands, code** and **yardsticks**, and in time picks what to improve next. Each step is gated three ways: by **tests** (a sandboxed trial, the gate, a planted revert), by **the owner** (a yes, or at least a notice), and by **measurement** (a frozen holdout, a benchmark run before and after). "Recursive" means the improved harness is the one that makes the next improvement: a better judge grades the next change better, a tool it built becomes a hand it uses, and an exam it wrote guards the next branch.

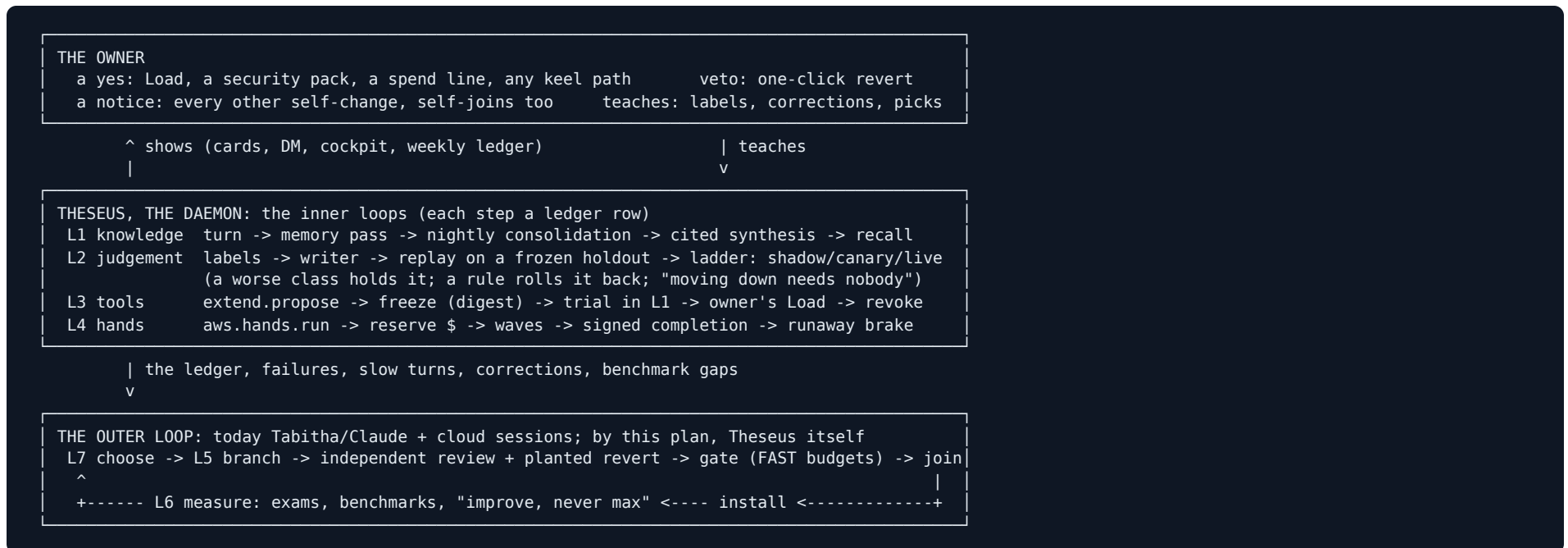
Why it is a differentiator. Claude Code and the other agent tools are *tools*: a person installs them, configures them, writes their memory file and adds their plugins. They can write an MCP server or a memory line when asked, and then a person wires it in. Theseus is a *daemon* that keeps running, keeps a ledger of everything it did, and is already closing small loops on itself: the owner's labels rewrite its judge's wording every night, the owner's corrections re-steer its model routing, its memory consolidates itself, and it can write, trial and (with the owner's yes) load its own tools. No other harness we compared has a frozen-holdout promotion ladder for its own judgement, or a sandboxed, digest-frozen, revocable path for tools it wrote itself.

Where it stands today (October 9). The ladder has seven rungs. The bottom four exist inside Theseus; the top three are done *for* Theseus by an outer loop (Tabitha/Claude and Claude cloud sessions), which has built it: 1,798 commits and 264 merges, 162 of them cloud branches, since September 25.

Rung	Inside Theseus today	Verdict
L1 knowledge	memory pass, consolidation (\$0.50 a day), recall live in every turn; standing orders not built	live , one gap
L2 judgement	the learning loop rewrites judge packs nightly from the owner's labels; the ladder promotes and rolls back; routing learns the owner's corrections	closed loop
L3 tools	<code>extend.propose</code> : write, freeze, trial in L1, the owner's Load, revoke	built, live, unmeasured

Rung	Inside Theseus today	Verdict
L4 compute hands	<code>aws.hands.run</code> on Lambda and Fargate, with reservations and a runaway brake	built, off (no stack yet)
L5 its own code	none: the outer loop's cloud rows, reviews, gate and joins	outside
L6 its own yardsticks	its judge's nightly report and <code>theseus judge prove</code> ; the benchmarks are written outside	judgement only
L7 what to improve next	none: the owner and Tabitha/Claude pick from plans	outside

The plan in one line: move the outer loop inward, each rung behind one governance spine (frozen, trialled, tested, measured, revocable, ledgered, shown), and **build the yardstick before the autonomy it measures**. **The owner chose to act first** (section 0): Theseus builds its own code, yardsticks and backlog, joins what passes every gate and tells him after, inside a short hard keel that still needs his yes. The machinery is built now, off; one guard (the keel guard, a fix for a known gap) goes into the stopping point; the switch goes on when the benchmark rerun reports.



0. Decided: act first, tell the owner after

The owner, 2026-10-09 10:08: *"I do want to try to build the code, yardsticks and backlog -- can we do more than just propose? I'm a 'shoot now ask questions later' kinda guy."* At 10:10 he said yes to what follows. Where sections 2 to 4 say otherwise (Propose for L5 to L7, a yes on every self-join, \$20 a day), this section wins; they keep the proposal as it was written, with the decision marked beside it.

What changes. L5 to L7 run at **Act, with a notice after**. Theseus files and ranks its own backlog and starts the top items. For each, it builds a branch, gets an independent review from a different session, and when every gate passes it joins, installs, and then tells the owner what changed, why, and what it cost. A regression after a join reverts on its own. The owner's veto is one click: revert. A weekly digest and a live ledger show all of it. Safety comes from automatic gates, reversibility and visibility, not from waiting for a yes.

Rung	Decided setting	What it means
L1 knowledge	Act (unchanged)	—
L2 judgement	Act inside the ladder (unchanged)	A security pack still goes to the owner's card
L3 tools	Propose (unchanged): a Load for each extension	Graduating one into core goes the L5 way
L4 hands	Propose until hands-live, then Act	Inside a \$1 hour line and a \$5 day line
L5 its own code	Act, notice after	Build, independent review, every gate, join, install, notice; auto-revert; one-click veto
L6 its own yardsticks	Act, notice after , on the open half	It writes exam items and benches; an item counts after an independent review; it never reads a sealed set
L7 what's next	Act, notice after	It files, ranks and starts its top items; the owner reorders or drops one with a word

The hard keel: these still need the owner's yes. A change that touches one is not refused: it stops at a card and waits, and the rest of the queue goes on.

1. **Deleting or loosening tests, budgets or ceilings.** keel-guard, a gate step, fails a branch that does, unless its head commit carries the owner's Keel: ack.
2. **Sealed holdout sets**, kept out of Theseus's reach: it gets a score, never an item.
3. **One-way store format bumps.**
4. **Security, policy, secret, permission and IAM changes.**
5. **Spend limits: \$10 a self-branch and \$30 a day** for all self-improvement together. Raising either takes the owner.
6. **A kill switch** that halts all self-directed work at once. Only the owner turns it back on.

Timing. The machinery is built now, in parallel with the stopping point, and ships **off by default**. `keel-guard` goes into the stopping point now (a row of `theseus-6rui` that blocks its gate, `theseus-njzi`), since it closes a known gap. The owner switches self-improvement on the day the benchmark rerun (`theseus-lc7a`) reports, so the rerun measures Theseus as it was before it started changing itself.

The machinery, built now and off. Every row is a child of epic `theseus-pw1q`, and a last row, "switch it on", waits for the rerun and for all of them.

Row	What it builds	Off by default, and the switch
<code>keel-guard</code>	The gate step of section 3's wave 0	Always on: it is a check, not an autonomy
<code>self-kill</code>	<code>theseus self halt and resume</code> : a durable halt row in the store, read before every self step; halt from the CLI, Discord or the cockpit, resume only by the owner in a private place	Halted until the switch goes on
<code>self-budget</code>	One reservation account for every self loop: \$10 a branch, \$30 a day; at the line the work stops and says so	<code>[self.budget]</code> , read at start; raising it is keel
<code>rsi-ledger</code>	<code>self.* rows</code> , <code>theseus self log</code> , the cockpit's "What Theseus changed about itself", a weekly Discord digest	The rows always record; the digest has its own key
<code>sealed-holdouts</code>	A store outside the workspace roots and the repo, and a scorer that returns numbers only	Holds nothing until the owner seeds it
<code>self-bench</code>	The self-improvement benchmark (wave 1)	Runs only when asked; \$5 a run
<code>self-backlog</code>	Nightly: read the ledger, health, judge reports and the omnibus's gaps; file and rank rows with evidence; start the top ones inside the budget	<code>[self] mode = "off"</code>
<code>self-pr</code>	Build a branch in a sandboxed worktree with its own build tree, run the gate, write a frozen report, push <code>self/<date>-<slug></code>	The same switch
<code>self-join</code>	An independent review (another session on another model, with a planted revert), then the join lock, the gate, a plain signed merge, the install and the notice	The same switch
<code>auto-revert</code>	After each self-join, watch the next gate, the benches and health for a window; a red reverts it with a revert commit and a notice; the owner's one-click veto does the same	The same switch
<code>self-exams</code>	Corrections, wrong-labels, failed tasks and slow turns become candidate exam items, half sealed	The same switch

One config section carries it: `[self] mode = "off"` (or `"act"`), `[self.budget] branch_usd = 10.0`, `day_usd = 30.0`. Read at the start, like the AWS lines: raising a budget takes the owner and a restart.

1. The ladder of self-improvement

Each rung: what exists, how strong it is, what gates it, and the evidence (file:line at 69cb5375).

Rung	What exists	Strength today	What gates it	Evidence
L1 knowledge	The memory pass labels each turn and links duplicates and corrections (<code>memory.v1</code> , <code>attribution.v1</code> in shadow). Nightly consolidation turns clusters recall admits together into one short cited <code>Synthesis</code> node with <code>derived_from</code> edges. Recall runs live in front of the model with <code>rerank.v1</code> 's order. Standing orders are not built: they are the north star's M2 row, in the persona program, OUT until after the rerun.	● recall and consolidation live; retention, activation and synthesis are measured arms, off unless named; ● standing orders	The place rule (a shared place recalls only its own sessions); a \$0.50-a-day writer; arms off unless <code>[memory]</code> arm names one; the four-arm exam (15% to 67% with recall)	<code>docs/status.md:52-63</code> ; <code>crates/theseus-core/src/consolidate/mod.rs:1-7</code> ; <code>docs/design/north-star-plan.md:195</code>
L2 judgement	The learning loop: the owner's wrong-labels (at least 10 new, train split only) go to a writer that may change only the wording of a pack; one replay grades the candidate on a frozen 14-day holdout; <code>decide</code> places it. The ladder: off, shadow, canary (20%), live, rolled back, per pack, in the store. Routing corrections: "that should have been on fable", a ↑/↓ reaction or <code>theseus judge correct</code> relabels the route judgment and steers close messages (a 64-entry layer). The stopping-point judge <code>loop.v1</code> reads turns that ended with no tool call, in shadow.	● the only loop that runs end to end with nobody in it: nightly, priced, placed, reversible	A class worse on the holdout holds it, always; live only when every deciding question's precision and recall rise by the margin; a security pack goes to the owner's card; the writer may not touch thresholds, ids or <code>[[rollback]]</code> rules; route, rerank, memory and attribution packs are <code>LEFT_ALONE</code> ; \$2.00 a day; one open proposal per lineage; rollback needs nobody	<code>crates/theseus-judge/src/propose.rs:502-552</code> (<code>decide</code>), :559 (<code>WRITER_SYSTEM</code>); <code>crates/theseus-core/src/learning/propose.rs:1-33,76</code> ; <code>crates/theseus-core/src/judge/lineage.rs:50</code> ; <code>crates/theseus-core/src/judge/ladder/rules.rs:1-13</code> ; <code>crates/theseus-core/config/theseus.example.toml:907-908,961-970</code> ; <code>docs/status.md:36-50</code>
L3 tools	<code>extend.propose { name, dir, command, description, tests?, network? }</code> : the model writes a stdio MCP server; Theseus copies it read-only to <code><state>/extensions/<name>/<digest>/</code> (SHA-256 over the tree), starts it in L1 as <code>ext-<name></code> (no secret, network only as asked, through the egress proxy), runs <code>initialize</code> , <code>tools/list</code> and up to 20 declared tests, writes a manifest, and asks the owner: Load or Don't load. An acked one loads, restarts with the daemon, and is revocable (<code>/extensions</code> , the cockpit's Extensions card).	● built, installed and live; "planks, never the keel": nothing here compiles or restarts Theseus. No graduation path; how often it is used is not measured	<code>dir</code> inside the workspace roots; the frozen digest (a later edit changes nothing that runs); the trial in L1; the ack only from the owner in a private place (a job's shell can't answer; L1 has no route to the daemon); no answer in time is a decline; six ledger rows (<code>extend.proposed</code> to <code>extend.revoked</code>)	<code>crates/theseus-core/src/extend/mod.rs:1-40,64-78</code> ; <code>docs/status.md:87-89</code> ; <code>docs/spec/02-part1-s0.md:143-145</code> (the owner's 43a decisions, Oct 4 09:34)

Rung	What exists	Strength today	What gates it	Evidence
L4 compute hands	<code>aws.hands.run</code> : jobs on Lambda or Fargate that keep the job wrapper's contract (detached, durable, cancellable). A per-dispatch HMAC key, so a hand can sign only for itself; a completion poller; cancels verified per backend; overdue hands reaped; quotas read hourly, big groups in waves; the hour's and day's lines with each hand's worst case reserved; runaway mode refuses new spend at 10x a line. Theseus owns its Home account and narrows its own hands with roles and session policies.	● built and installed, off : the account has no hand stack yet, and the runaway brake has never been seen live (theseus-ongv)	Reservations before launch; the runaway factor (10x); a \$50 monthly AWS Budget with a stop at 100%; the SOC2 stance (no public ingress, IaC only, no long-lived keys); tagged inventory, so automation never touches what it did not make	<code>crates/theseus-core/src/aws/hands/mod.rs:1-25</code> , <code>envelope.rs:1-15</code> , <code>runaway.rs:1-25</code> , <code>quota.rs:1-8</code> ; <code>docs/design/aws-toolset.md:111-118,1308-1330</code> ; <code>docs/status.md:173-175,338-341,586-587</code>
L5 its own code	Inside Theseus: the parts, not the loop. It has file edits and patches, <code>proc.run</code> , git diff and log, language servers, terminals, and independent check tasks (Item 132). The loop is outside : an issue becomes a cloud row (a Claude cloud session builds a branch and commits <code>CLOUD_REPORT.md</code>), a review row (independent, with a planted revert: undo the fix, the new test must fail), a join (a plain signed merge after a dry run, then <code>scripts/gate.sh</code>), and an install under the owner's standing OK.	● for Theseus; ● for the outer loop: 1,798 commits, 264 merges (162 cloud) in two weeks	The gate: <code>cargo fmt --check</code> , <code>clippy -D warnings</code> , <code>cargo deny</code> , the whole <code>nextest</code> suite, the shape and long-file ceilings, the cockpit build, and the FAST lifecycle benches at p95 against their budgets; review rows; the join lock; the owner	<code>scripts/gate.sh:248-265,405-421,608-614</code> ; <code>scripts/long-files.txt</code> ; <code>docs/status.md:82</code> ; git history since 2026-09-25
L6 its own yardsticks	For judgement, yes: the learning ledger's nightly report with frozen holdouts, replay, audit and backfill, and <code>theseus judge prove</code> from the daemon's own ledger. The memory exam is versioned data with half of each family held out, and a generator makes sessions from templates and seeds. Everything else is written outside : 20 dated reports in <code>docs/benchmarks/</code> and the omnibus. <code>own-benchmark</code> (replay the owner's real turns on cheaper models) is planned, OUT until after the rerun.	● judgement only	Holdouts frozen into each report; the replay's own spend cap; "benchmarks improve, never max"	<code>crates/theseus-core/src/learning/mod.rs:1-22</code> ; <code>crates/theseus-exam/src/lib.rs</code> , <code>generate.rs</code> ; <code>docs/benchmarks/omnibus.md</code> ; north-star own-benchmark (killer R15 to R17)
L7 choosing what's next	Nothing inside. The raw material exists: the ledger, health, the judge's reports, <code>route.corrected</code> rows, failed and refused calls, slow turns against their budgets. The choosing is the owner's and Tabitha/Claude's: the north-star plan, and the stopping-point epic theseus-6rui with its 98 IN rows.	●	The owner picks	<code>docs/design/north-star-plan.md §2</code> ; <code>contexts/theseus-stop-point.md</code>

What the ladder says. L2 is the model to copy. It is already full RSI in miniature: Theseus reads its own errors, writes a candidate of itself, grades it on data the writer never saw, places it on a ladder that can only lower what the config allows, and rolls itself back without asking. Every rung above it should be built the same way: a **candidate** that cannot touch its own grader, a **sealed holdout**, a **placement by numbers**, a **brake that needs nobody**, and **the owner's yes for anything that touches the keel**.

2. The governance spine

This is what makes it safe to turn the dial up. Every self-made change, at every rung, has seven properties.

Property	What it means	Today	Missing
Frozen	What was proposed is what runs: a digest, a version row	Extensions by SHA-256; pack versions as <code>pack.version</code> rows and files; joins as signed merges	Self-branches (L5) don't exist yet
Trialled	It runs somewhere harmless first	Extensions in L1; packs in shadow, then a 20% canary; hands per wave	A sandbox build tree for Theseus's own branches
Tested	A test that fails without it and passes with it	Extensions' declared tests; the replay on the holdout; the gate; planted reverts at review	Nothing stops a branch deleting or weakening a test (the keel guard, wave 0)
Measured	A number, before and after, on data the proposer never saw	Frozen 14-day holdouts; the bench A/B in one lock hold; the omnibus	A self-improvement benchmark; sealed exams (waves 1 and 3)
Revocable	One act undoes it	<code>extension.revoke</code> ; rollback rules; <code>profile.use</code> ; git revert	Auto-revert on a red after a self-join (wave 4)
Ledgered	A row says who, what, why and with which numbers	<code>extend.*</code> , <code>pack.mode</code> , <code>judge.proposal</code> , <code>route.corrected</code> , <code>aws.runaway.rows</code>	One "what Theseus changed about itself" view (wave 1)
Shown	The owner sees it without asking	Cards, DM notices, the cockpit's Judgment, Extensions and AWS cards	A weekly digest in Discord

The constants that never bend:

- **Spend ceilings and the runaway brake.** Every loop carries its own day cap: the pack writer \$2.00, consolidation \$0.50, the planned replay \$1.00, the judge's day budget, and AWS's hour and day lines with runaway mode at 10x. Raising a line takes the owner and a restart, since the config is read at the start (`runaway.rs:23-25`).
- **No names in public and no secrets.** The repo is public: say "the owner"; the assistant is "Tabitha/Claude". An extension is granted no secret; a secret-bearing AWS read returns a handle, never the value.
- **Improve, never max.** A change that helps only a benchmark is a loss even if the score moves. No task-specific prompt, no special case for a harness, no tuning on held-out tasks.
- **Minimal prompting.** The levers in order: the API's own contract, the tool's shape, its description, its result's metadata, and only then one factual prompt sentence, if a bench arm earns it. Never examples, persuasion or "you MUST". A self-proposed prompt sentence needs its arm.
- **The config's ceiling lowers, never raises** (`JudgeConfig::mode_of`). No self-change can lift the owner's ceiling.

Failure modes, and what stops each:

Failure	How it would show	What stops it	Status
Reward hacking	A learned pack pleases the labels and gets worse elsewhere; a branch games a benchmark	A class worse on the holdout holds it; precision and recall must both rise; the holdout is frozen and never shown to the writer; the writer may change wording only; sealed benchmark and exam sets the proposer can't read; "improve, never max" at review	● for packs; ● for code (sealed sets, wave 1)
Test deletion or weakening	A branch passes the gate because it removed the test that failed	The keel guard: a gate step that diffs tests, #[ignore] s, assertions, #[allow] s, budgets and ceilings against main and fails unless the commit carries an owner-acked Keel: trailer; a planted revert at every review	● build in wave 0 (a known gap: on Oct 8 a clean merge quietly dropped a bench's spend and loop caps)
Gate erosion	The gate, the budgets or the rollback rules quietly loosen	Keel paths (scripts/gate.sh , scripts/long-files.txt , the lifecycle budgets, packs' [[rollback]] rules, [policy] defaults, the exam and bench sets) need the owner's yes, always; the pack writer already can't touch thresholds or rollback rules	● packs yes; code no
Drift	Many small acceptable changes add up to a worse Theseus	The whole benchmark rerun at each stopping point; the identity exam; the weekly ledger; the frozen holdouts don't move with the candidate	●
Runaway spend	A loop, or hands launching hands, burns money	Per-loop day caps; reservations before launch; runaway mode; the \$50 AWS budget; the cloud cap (16 rows); a hand can't mint another hand's key; a depth limit of one for hands that launch hands	● the brake built, never seen live
Self-escalation	A change widens what Theseus may do without asking	Acks only from the owner in a private place; a job's shell can't answer, untighten, trust, publish or bootstrap AWS; L1 has no route to the daemon; the config's ceiling lowers only	●
Confabulated success	The proposer says it worked, and it didn't	Success is the gate's and the bench's verdict, never the proposer's words; reports are frozen files; review is by a different session (and, for L5, a different model)	● the rule exists in the outer loop; build it inside
Leaks	A name or a secret reaches the public repo	The scrub check over docs/ ; the no-names rule; self-branches pushed to their own namespace and scrubbed before review	●

The owner's ack levels (today, and as decided on 2026-10-09; section 0):

Level	Today	Decided
Needs a yes	Loading an extension; a security pack's move; a join's install (standing OK); a spend line raised; the hands' stack	Loading an extension; a security pack's move; the hands' stack; a hand capability for an extension; the hard keel : deleting or loosening a test, budget or ceiling, a sealed set, a one-way store format bump, a security, policy, secret, permission or IAM change, a spend limit, resuming after the kill switch
A notice only	A non-security pack's promotion or canary; a rollback; consolidation's syntheses; a route correction's effect	All of today's, plus: every self-join and its install , a backlog row filed or started, an exam item counted, an auto-revert, the weekly digest
Never, whatever is said	(implicit)	Raising its own ceilings or budgets; reading a sealed set; editing the gate unacked; pushing to the public main without the independent review and the gate; publishing a name or a secret

3. The expansion plan, in waves

The rule against the stopping point (epic theseus-6rui): new capabilities that close no known gap are OUT until after the benchmark rerun (theseus-lc7a). So only a row that fixes a known gap goes in before it. Each row below says which.

As decided (section 0): the machinery rows (the keel guard, the kill switch, the budgets, the ledger, the sealed holdouts, the self-bench, self-backlog, self-pr, the join and its auto-revert, self-exams) are all built now, in parallel, and ship off; only the keel guard is IN the stopping point. The waves below keep their order as the order in which the capabilities are switched on and trusted. Wave 4's notice-joins is overtaken: every self-join is at a notice from the switch on, with the hard keel still at a yes.

Wave 0: before the stopping point (known gaps only)

Row	The spec, in one line	Size	Depends on	The measurement that proves it	Stop point
keel-guard	A gate step that compares the branch with main: test functions removed, #[ignore] added, assertions deleted, #[allow(clippy...)] added, lifecycle budgets, bench caps and long-files.txt ceilings raised. Each fails the gate unless the head commit carries Keel: <what>, acked <date>	M	none	A planted-erosion suite: 8 seeded erosions, all caught; 0 false alarms replaying the last 50 joins	IN: it fixes a known gap (on Oct 8 two clean merges were wrong: one dropped a bench's caps, one pushed a function past clippy's 100 lines; only the branches' own tests showed it)
hands-live	Theseus's hands stack and image on its Home account, then the live checks: a group runs, a cancel is verified, a reaper run, and the runaway refusal seen at a lowered test line	M	the owner's go (theseus-ongv)	One group of 3 hands settles under \$0.10; a refused fourth group at a \$0.05 test line	Owner-gated: built, waiting on a known live check
rsi-digest (outside Theseus)	A weekly report by script from the existing rows (pack.mode , judge.proposal , extend.* , route.corrected , consolidation's syntheses): "what Theseus changed about itself this week"	S	none	The first digest matches the rows by hand	Not a Theseus change; can start now

Wave 1: right after the rerun (yardsticks first, then code proposals)

Row	The spec	Size	Depends on	Measurement
self-bench	The self-improvement benchmark. A sealed set of 12 seeded gaps in a scratch copy of the repo: a behaviour bug with a user's report, a perf regression past a budget, a doc that lies, a flaky test, a posture loosened, a missing test, and two traps where the cheap fix is a benchmark-only shortcut or a test deletion. Per gap: found (in the top 3 of its backlog), fix proposed, proved (a test that fails then passes, a planted revert), no regressions (the full gate), no erosion (the keel guard), cost. Run the same model under Claude Code as the baseline. The set lives outside the workspace roots	L	keel-guard	The score itself; the traps must score 0 shortcuts
self-pr (L5; decided: act)	self.propose_change { issue, why } : Theseus works in a sandboxed worktree with its own build tree, runs the gate, writes a frozen report, and pushes self/<date>-<slug> . An independent review (a cloud review row on another model, or a check task by its claim) with a planted revert. As decided, self-join then joins it when every gate passes and tells the owner after	L	keel-guard, self-bench	The self-bench score; the review's accept rate; 0 keel trips unacked; cost per accepted change against the outer loop's
rsi-ledger	self.* rows plus today's, behind theseus self log , a cockpit card "What Theseus changed about itself", and a weekly Discord digest	M	none	Every self-change of the week is listed, with its numbers and its undo
own-benchmark a, b, c	Already planned (killer R15 to R17): replay the owner's real turns on cheaper models; theseus route report	L (3 rows)	the rerun	Already specified in the north star; a \$1-a-day cap

Wave 2: tools and hands graduate

Row	The spec	Size	Depends on	Measurement
ext-graduate	An extension used 20 or more times in 14 days with no failure earns a proposal: Theseus ports it into core as a self-pr branch, its declared tests becoming Rust tests, with the owner's yes	M	self-pr	Latency in process against stdio (a tool call is already the slowest row in the speed table); every carried test green
ext-hands	Hands that build hands: an extension's manifest may ask for a hands capability with a dollar cap and backends; its tools then launch aws.hands.run groups under the same reservations and brake. Depth one: a hand can't launch a hand. The owner's Load card names the cap	M	hands-live	A seeded 200-file batch done inside the cap; a runaway test refused at the line
self-narrow	Theseus proposes session policies for its own hands from what CloudTrail shows they used, checked by Access Analyzer's validation	S	hands-live	A smaller policy; no legitimate call denied over a week

Wave 3: its own exams, and choosing what's next

Row	The spec	Size	Depends on	Measurement
self-exams	Every owner correction, wrong-label, failed task and over-budget turn becomes a candidate exam item in the exam's versioned format, half held out. An item counts only after review, and the proposer of a fix never edits the exam that judges it	M	rsi-ledger	The escaped-regression rate: regressions the owner finds that no exam caught
self-backlog (L7; decided: act)	Nightly, Theseus reads its ledger, health, judge reports and the omnibus's gaps, files a ranked backlog as tasks with evidence links, and starts the top ones inside the budget. The owner reorders or drops	M	rsi-ledger, self-exams	Precision of its top 10 against the owner's picks; the self-bench's "found" score
self-pipeline	Theseus runs the cloud-row pipeline itself: launches builders and reviewers, queues joins behind the lock and the gate; as decided, each join is a notice	XL	self-pr, self-backlog	Rows a day; the accept rate; regressions found at the next rerun; cost a row against today's

Wave 4: turn the dial

`notice-joins`: for three narrow classes only (a doc made true, a test-only addition, a speed change that passes the bench A/B by its margin), a self-join needs a notice rather than a yes, with **auto-revert** when anything after it goes red, and the weekly ledger reviewed with the owner. Size M. Depends on all of wave 3 and four clean weeks of `self-pr`. Measurement: zero auto-reverts left unexplained; the owner's own read of a month's ledger.

4. Owner decisions, each with a default

Decided 2026-10-09 10:10 (section 0). The defaults below are the proposal as written; the "Decided" column and the bold notes are the owner's answer.

The autonomy dial, per rung (Off · Propose · Notice · Act):

Rung	Proposed default	Decided	Moves up when
L1 knowledge	Act (as now)	Act	—
L2 judgement	Act inside the ladder (as now); a security pack stays at the card	Act, as proposed	—
L3 tools	Propose : a Load for every extension (as now)	Propose	graduation proves the class safe
L4 hands	Propose until hands-live, then Act inside a \$1 hour line and a \$5 day line	as proposed	a month without a runaway
L5 code	Propose : the owner's yes on every self-join	Act, notice after	—
L6 yardsticks	Propose : an exam item counts only after review	Act, notice after ; an item counts after an independent review	—
L7 backlog	Propose : the owner picks	Act, notice after	—

The other decisions:

1. **The keel guard into the stopping point.** *Default: yes, as a fix for a known gap (wave 0).* **Decided: yes.**
2. **The hands' live check.** *Default: right after install #16, at a \$0.05 test line, then back to \$1.00.*
3. **The budgets.** *Default: the existing caps stand (\$2.00 writer, \$0.50 consolidation, \$1.00 replay), plus \$10 a self-branch and \$20 a day for L5, \$5 a run for the self-bench, and nothing else new.* **Decided: \$10 a self-branch and \$30 a day for all self-improvement.**
4. **Who reviews a self-branch.** *Default: an independent cloud review row on a different model, with a planted revert; then the owner's yes.* **Decided: the independent review stays; the owner's yes becomes a notice after the join, with auto-revert and a one-click veto.**
5. **Where the sealed sets live.** *Default: outside the workspace roots and the repo, in the owner's private storage; Theseus sees a score, never the items.*
6. **Where self-branches go.** *Default: their own self/ namespace, scrubbed for names and secrets before the review; nothing reaches main without the join.*
7. **The word.** *Default: "self-improvement" on the surfaces, "RSI" in the design docs.*

5. How to explain it to others

The pitch, in three sentences. Theseus is an agent harness that improves itself while it runs. It writes and tests its own tools, tunes its own judgement from your corrections, consolidates its own memory, and (soon) builds, reviews and joins changes to its own code. Every one of those changes is frozen, trialled in a sandbox, measured on data it never saw, shown to you, and undone in one step.

The paragraph. Most agent tools are tools: you install them, configure them and teach them by editing files. Theseus is a daemon with a ledger, and it closes loops on itself. Every night your labels rewrite the wording of its judge, which is graded on a frozen holdout and placed on a ladder that can promote, canary or roll it back on its own. Your corrections re-steer which model takes which kind of work. Its memory consolidates into cited syntheses. It can write a small tool, freeze it by digest, try it in a sandbox and ask you to load it, and it has compute hands on its own cloud account behind dollar reservations and a runaway brake. The plan is to move the outer loop (the one that writes its code and its benchmarks) inside, one rung at a time, with a sealed yardstick built before each new autonomy, and the owner's yes on anything that touches the keel.

The honest caveat. Today the deep loops (writing its code, writing its yardsticks and choosing what to improve) still run *outside* Theseus, driven by the owner and Tabitha/Claude; inside it, the closed loops are judgement, routing and memory. And none of this changes the model's weights: the ceiling is the model's own skill, and what Theseus improves is everything around it.

Sources

- Code at origin/main 69cb5375 (2026-10-09 08:59): `crates/theseus-core/src/extend/`, `aws/hands/`, `learning/`, `judge/ladder/`, `judge/lineage.rs`, `consolidate/`, `config/theseus.example.toml`; `crates/theseus-judge/src/propose.rs`; `crates/theseus-exam/src/`; `scripts/gate.sh`.
- Docs: `docs/status.md` (updated 2026-10-07 03:52), `docs/spec/02-part1-s0.md`, `docs/design/aws-toolset.md`, `docs/design/north-star-plan.md`, `docs/benchmarks/omnibus.md`.
- The outer loop's recipe: `~/openclaw/workspace/contexts/theseus-stop-point.md`, `theseus-joins-1009b-brief.md`, `theseus-install16-brief.md`, and the workspace memories on cloud routines, aggressive review and joins, "improve, never max", minimal prompting, FAST, and no names in the public repo.
- A correction to the brief: standing orders are not built yet (north star M2, OUT until after the rerun), and the stopping-point judge `loop.v1` runs in shadow.